The MABEL Control Stack: A Flowchart Atlas

Every layer from operator device to actuator, who subscribes to what,
and how the motion model, differential controllers, and retargeting compose
(`controller/mabel` and its neighbours)

MABEL Project — Control & Teleoperation

November 2, 2026

Abstract

This document is the single-page-per-question visual reference for MABEL's control stack. Fig. 1 lays out the *entire* command, state, video, ROS, and map paths — from the seven operator frontends (Vision Pro v1/v2, iOS v1/v2, web teleop, Control Center, Nav2 autonomy) through the one wire contract, the gateway, the `mabel` control library, the pluggable plant seam, the ROS 2 DDS transport, and the hardware abstraction layer (HAL) down to the physical devices — and marks the two load-bearing invariants: the HAL is the *sole* hardware owner and `controller/mabel` is the *sole* gate on anything that moves the robot. Section 2 tabulates exactly who subscribes to what. The remaining figures zoom into the controller itself: the per-tick pipeline of the `MabelTeleop` facade and `TeleopEngine` (Fig. 2), the lift-aware tip-over motion model `safe_base_step` (Fig. 3), the differential IK and torque-domain whole-body controllers (Fig. 4), and the hand / whole-body retargeting chain (Fig. 5). Derivations are not duplicated here; each section cites the sibling technical paper that owns them.

Keywords: teleoperation, whole-body control, differential inverse kinematics, damped least squares, ZMP tip-over safety, retargeting, swerve drive, system architecture.

1 Overview and reading guide

MABEL's control stack is deliberately a *single* pipeline with pluggable ends: every input device speaks one wire contract, every command funnels through one control library, and one hardware seam decides whether the identical `RobotCommand` lands in MuJoCo or on the real robot. The stack is easiest to hold in mind as five layers plus three side paths:

1. **Devices** (§2) — iOS, Vision Pro, web, Control Center, Nav2. All are *clients* of one WebSocket wire contract; none may publish a command topic directly.
2. **The gateway** — `server/sim_teleop_bridge.py` (it serves the real robot too, despite the name): connection handling, driver arbitration, security. Pure glue; the control logic lives in the library below.
3. **The controller** — the installable `mabel` package (`controller/`), the sole gate on motion: retargeting, whole-body IK, torque engine, tip-over motion model, fail-safes (Fig. 2).
4. **The plant seam** — `server/plant.py`: `SimBridgePlant` (RPC to the MuJoCo `simulation_bridge`) or `ROSPlant` (ROS 2 DDS to the HAL). `RobotCommand` is byte-identical either way.
5. **The HAL** — `hardware_bridge/` (`mabel_hw`) on the thor Jetson: the only process allowed to open a device.

The side paths: **video** flows HAL → devices directly (never through the gateway); **ROS** (rviz, SLAM, Nav2) is a thin DDS *subscriber* of HAL state; the **map** rides SLAM → gateway → devices. These invariants are enforced by the HAL contract tests (`hardware_bridge/tests/`).

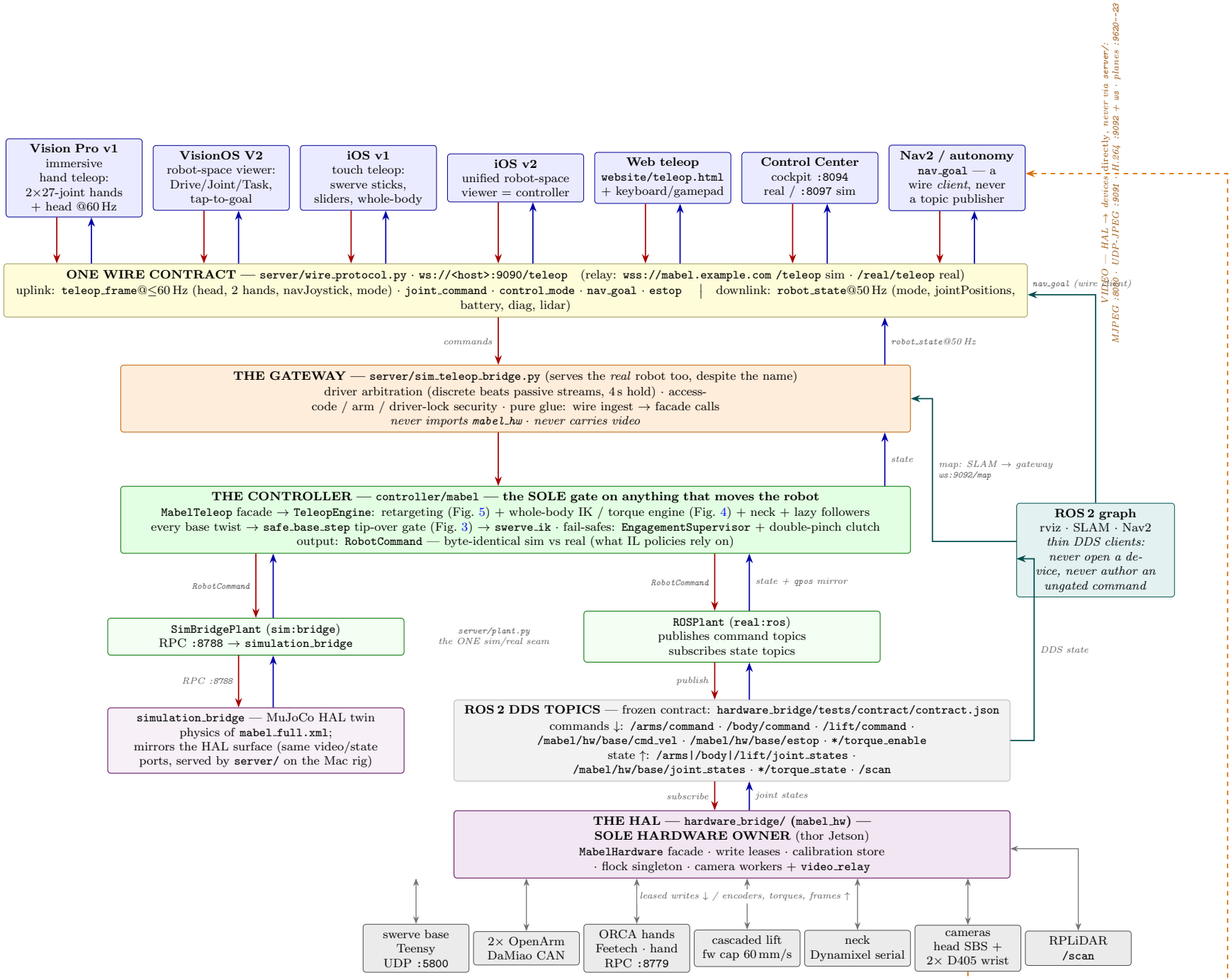
2 The full stack and its subscribers

Fig. 1 is the master flowchart. Command flow (red) runs top to bottom on the left of each layer; state flow (blue) returns bottom to top on the right; video (orange, dashed) and the ROS/map paths (teal) bypass the layers they are forbidden to touch. Table 1 is the same information indexed by consumer: for each process, what it subscribes to, what it emits, and over which transport.

Table 1: Who subscribes to what. "Wire" is the WebSocket contract of `server/wire_protocol.py` (`ws://<host>:9090`); DDS topic names are the frozen HAL contract (`hardware_bridge/tests/contract/contract.json`).

Consumer	Subscribes to / consumes	Emits	Transport
Apps: iOS v1/v2, VisionOS v1/v2, web teleop, Control Center	`robot_state` @50 Hz; video MJPEG `:8080`, UDP-JPEG `:9091`, H.264 `:9092` + `ws :8080/camera/<n>/h264`, dedicated planes `:9620--23`; map `ws :9092/map`	`teleop_frame` @≤60 Hz, `joint_command`, `control_mode`, `nav_goal`, `estop`	wire (LAN or relay), UDP
Gateway `server/`	plant state (DDS joint states via `ROSPlant`, or RPC `:8788` via `SimBridgePlant`); wire uplink from all clients	`robot_state` broadcast; facade calls into `mabel`	wire + DDS/RPC
Controller `mabel`	`OperatorPose` (from `sources/`); robot `qpos` mirror (encoders each frame on hardware)	`RobotCommand` (actuator targets, byte-identical sim vs real)	in-process (composed by the gateway)
HAL `hardware_bridge`	`/arms/command`, `/body/command`, `/lift/command`, `/mabel/hw/base/cmd_vel`, `/mabel/hw/base/estop`, `*/torque_enable`	`/arms|/body|/lift/joint_states`, `/mabel/hw/base/joint_states`, `*/torque_state`, `/scan`; all video ports	DDS + device buses
ROS graph (rviz, SLAM, Nav2)	HAL state topics, `/scan`	map → gateway (`ws :9092/map`); `nav_goal` *as a wire client* — never a command-topic publisher	DDS; wire
IL recorder / policies	`robot_state` + `RobotCommand` pairs (identical sim/real is what makes the data transferable)	— (policies re-enter as wire clients)	—

Two rules of Fig. 1 bear repeating because every historical wiring bug violated one of them. First, *the HAL owns every device*: ROS nodes are DDS clients and the 2026-07 neck migration removed the last exception. Second, *every command source is a client of the one wire contract*, so arbitration, security, fail-safes and the tip-over gate apply uniformly — Nav2's autonomy goals enter as `nav_goal` wire messages exactly like a thumb on an iPhone joystick.


Vision Pro v1
immersive
hand teleop:
2×27-joint hands
+ head @60 Hz
VisionOS V2
robot-space viewer:
Drive/Joint/Task,
tap-to-goal
iOS v1
touch teleop:
swerve sticks,
sliders, whole-body
iOS v2
unified robot-space
viewer = controller
Web teleop
website/teleop.html
+ keyboard/gamepad
Control Center
cockpit :8094
real / :8097 sim
Nav2 / autonomy
nav_goal — a
wire client, never
a topic publisher
ONE WIRE CONTRACT — server/wire_protocol.py · ws://<host>:9090/teleop (relay: wss://mabel.example.com /teleop sim · /real/teleop real)
uplink: teleop_frame@≤60 Hz (head, 2 hands, navJoystick, mode) · joint_command · control_mode · nav_goal · estop | downlink: robot_state@50 Hz (mode, jointPositions, battery, diag, lidar)
nav_goal (wire client)
VIDEO — HAL → devices, directly, never via server/:
MJPEG :8080 · UDP-JPEG :9091 · H.264 :9092 + ws · planes :9620–23
commands
robot_state@50 Hz
THE GATEWAY — server/sim_teleop_bridge.py (serves the real robot too, despite the name)
driver arbitration (discrete beats passive streams, 4 s hold) · access-code / arm / driver-lock security · pure glue: wire ingest → facade calls
never imports mabel_hw · never carries video
state
map: SLAM → gateway
ws:9092/map
THE CONTROLLER — controller/mabel — the SOLE gate on anything that moves the robot
MabelTeleop facade → TeleopEngine: retargeting (Fig. 5) + whole-body IK / torque engine (Fig. 4) + neck + lazy followers
every base twist → safe_base_step tip-over gate (Fig. 3) → swerve_ik · fail-safes: EngagementSupervisor + double-pinch clutch
output: RobotCommand — byte-identical sim vs real (what IL policies rely on)
ROS 2 graph
rviz · SLAM · Nav2
thin DDS clients:
never open a device, never author an ungated command
RobotCommand
RobotCommand
state + qpos mirror
DDS state
SimBridgePlant (sim:bridge)
RPC :8788 → simulation_bridge
server/plant.py
the ONE sim/real seam
ROSPlant (real:ros)
publishes command topics
subscribes state topics
RPC :8788
publish
simulation_bridge — MuJoCo HAL twin
physics of mabel_full.xml;
mirrors the HAL surface (same video/state ports, served by server/ on the Mac rig)
ROS 2 DDS TOPICS — frozen contract: hardware_bridge/tests/contract/contract.json
commands ↓: /arms/command · /body/command · /lift/command · /mabel/hw/base/cmd_vel · /mabel/hw/base/estop · */torque_enable
state ↑: /arms|/body|/lift/joint_states · /mabel/hw/base/joint_states · */torque_state · /scan
subscribe
joint states
THE HAL — hardware_bridge/ (mabel_hw) — SOLE HARDWARE OWNER (thor Jetson)
MabelHardware facade · write leases · calibration store · flock singleton · camera workers + video_relay
leased writes ↓ / encoders, torques, frames ↑
swerve base
Teensy
UDP :5800
2× OpenArm
DaMiao CAN
ORCA hands
Feetech · hand
RPC :8779
cascaded lift
fw cap 60 mm/s
neck
Dynamixel serial
cameras
head SBS +
2× D405 wrist
RPLiDAR
/scan

3 Inside the controller: one teleop tick

Fig. 2 unrolls what happens between an `OperatorPose` arriving and actuator targets leaving. The facade `MabelTeleop.update(pose, dt)` (`mabel/facade.py`) composes three concerns, and the engine deliberately splits them by *rate*:

- **Per operator frame** ($\leq$60 Hz, memoized on the pose timestamp): the heavy retargeting — operator-frame construction, the extension-fraction arm map, elbow swivel, analytic Gauss–Newton arm IK ($\sim$5 ms), keypoint hand IK, and the decoupled neck look-at. The bridge calls `update()` once per physics substep (5–16$\times$ per frame); memoization is why that does not multiply the solve cost.
- **Per substep** (cheap physics): the lazy base/lift/torso followers or the nav joystick produce a target twist; it passes the tip-over gate `safe_base_step` and then `swerve_ik`; the engagement fail-safe blends everything toward safety when tracking drops. Gravity compensation is refreshed at 30 Hz (`MABEL_GRAV_HZ`) and persists across substeps.
- **Torque engine** (when the control spec selects a WBC method, `joint`/`wrist`, or a gravity-comp/compliance mode): region-masked impedance or compliance torques on top of the dynamics bias, filtered by the self-collision capsule guard (§5).

Every wire message maps onto a facade method (`control_mode` $\rightarrow$ `set_control_spec`, `joint_command` $\rightarrow$ `set_joint_targets`, `external_force` $\rightarrow$ `apply_external_force`, plus `recalibrate`/`reset`), so the gateway stays pure glue. Mode and region switches call `sync_to_current()` so control never snaps the arm.

4 The motion model: `safe_base_step`

The motion model (`mabel/motion_model/tip_over.py`; full derivation and validation in `motion_model.tex` and `wbc_stability.tex`) is the safety envelope between any base twist and `swerve_ik`. Braking at acceleration a with the centre of mass at height h tips the robot over the support edge at lever d_{dir} when $m\,a\,h > m\,g\,d_{\text{dir}}$, so the per-direction cap (mass cancels) is

$$a_{\text{tip}}(h) \;=\; g\,\frac{d_{\text{dir}}}{h}, \qquad h(z_{\text{lift}}) \;=\; \text{COM_Z0} + \text{COM_DZDLIFT}\cdot z_{\text{lift}}, \tag{1}$$

i.e. raising the lift shrinks the safe acceleration (and the derived speed ceiling $v_{\text{dir}} \approx a_{\text{dir}} T_{\text{stop}}$, with per-direction safety factors) like $1/h$. MABEL's triangular support polygon — two front wheels, one rear — and slightly forward CoM make *forward* the binding direction ($d_{\text{fwd}} = 0.078$ m vs $d_{\text{lat}} = 0.108$ m, $d_{\text{back}} = 0.241$ m, measured from the MJCF; agile strafing therefore comes from fast module steering, not high lateral speed). Fig. 3 shows the four-stage pipeline of `safe_base_step`$(v, v_t, dt, z_{\text{lift}})$: clip, project, ramp, clamp — each stage lift-aware, the last two enforcing the full *coupled* ZMP criterion (translation + rotation, centripetal terms included) rather than per-axis boxes. The constants are *measured* from `mabel_full.xml` and re-derived by `controller/experiments/wbc_stability.py` (E4) whenever robot geometry changes; the module is NumPy-only so the ROS navigation node imports it on the robot without MuJoCo.

5 The differential controllers

Two controllers share the "differential" label and Fig. 4 shows both; the full treatment is `whole_body_controller.tex`.

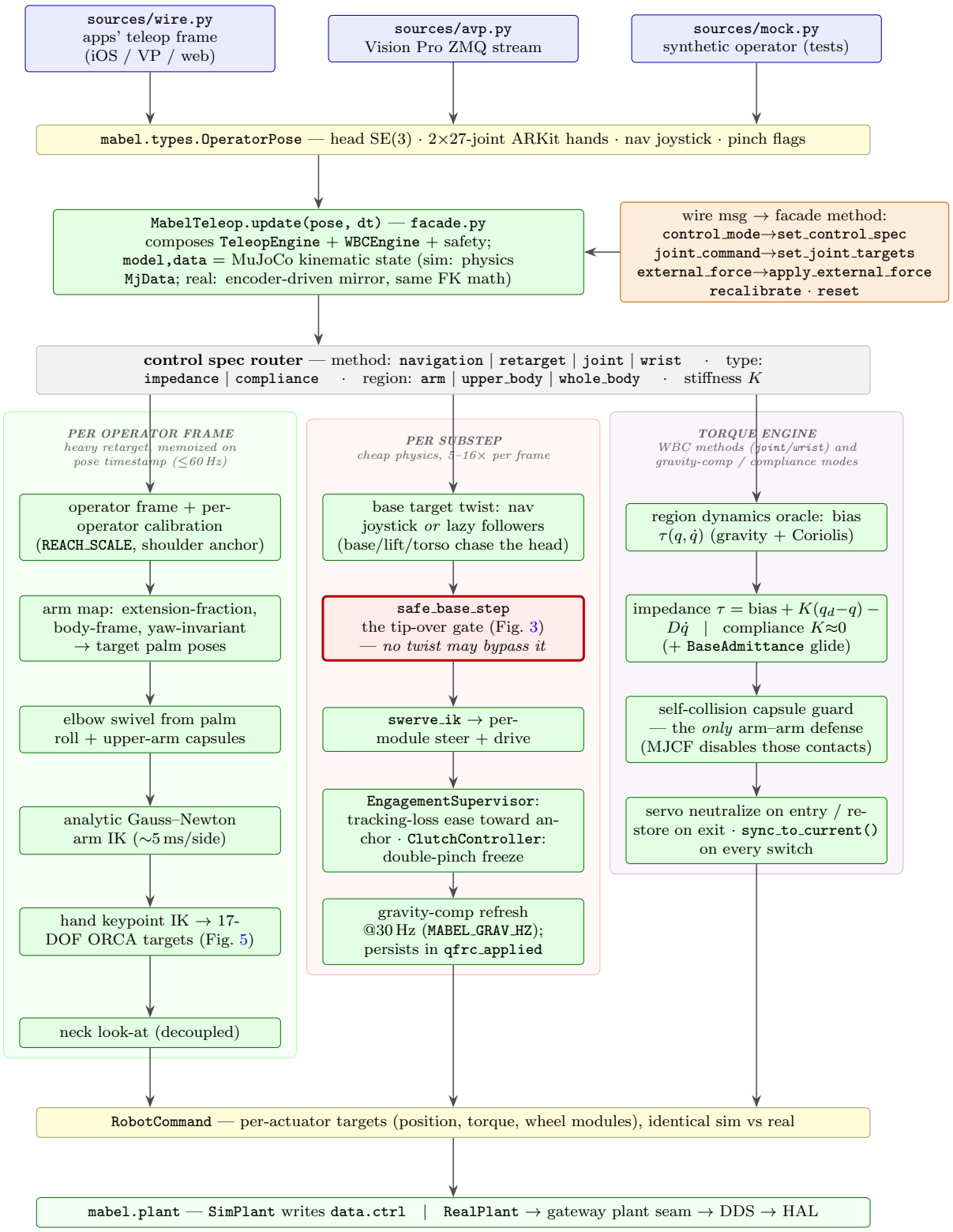


Figure 2: One controller tick: device ingest to `RobotCommand`. The three lanes run at different rates on purpose — heavy retargeting once per operator frame (memoized on the frame timestamp), cheap base/safety physics every substep, and the torque engine when the control spec engages it. The red box is the invariant: every base twist, from every source, passes `safe_base_step`.

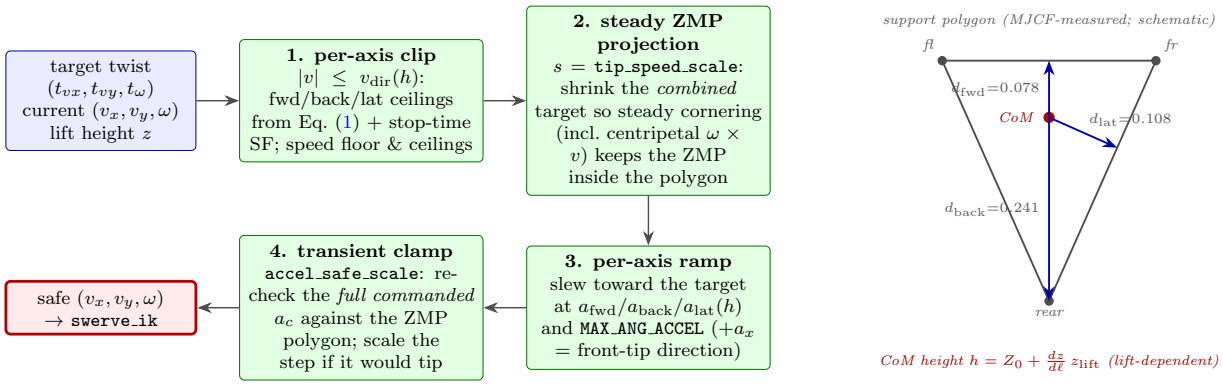


Figure 3: `safe_base_step`: clip → project → ramp → clamp, every stage a function of the lift-raised CoM height h. Right: the wheel-contact support triangle with the measured tip levers (metres) — the short forward lever is why forward braking is the binding constraint.

(a) Position domain — bimanual SE(3) differential IK. `whole_body_controller/ik.py` tracks two 6-DOF palm targets over the 18-DOF upper chain (lift + torso + per-side arm/wrist) with a weighted damped-least-squares step and a null-space posture task:

$$\dot{q} = \underbrace{\left(J^\top W J + \lambda^2 I\right)^{-1} J^\top W\, e}_{\text{weighted DLS}} + \underbrace{\left(I - J^{+} J\right) k_p \left(q_{\text{nom}} - q\right)}_{\text{null-space posture (HoMMI-style } C_{\text{nominal}})}, \qquad e = \begin{bmatrix} \Delta p \\ e_R \end{bmatrix}, \tag{2}$$

where e_R is the SO(3) column-cross error and a soft deadband on $\|e\|$ stops the arm chasing gravity sag when the operator is idle. Two implementation contracts matter: the lift is a tendon-cascaded pair, so the planner works in *joint* units, doubles the lift Jacobian column, and `apply()` writes ctrl $= 2\, q_{\text{lift}}$; and the integrated step is filtered by the capsule-guard QP before it reaches the actuators. The wrist-clutch path (`whole_body_retargeting/wrist_clutch.py`) reuses the same DLS machinery for clutched SE(3) wrist teleop.

(b) Torque domain — the `WBCEngine` runtime. `whole_body_controller/engine.py` is parameterised on two orthogonal axes: control *type* (impedance: $\tau = \text{bias}(q, \dot{q}) + K(q_d - q) - D\dot{q}$, rigid tracking; compliance: $K \approx 0$, gravity-comp + damping, back-drivable) and *region* (`arm` ⊂ `upper_body` ⊂ `whole_body`: the larger regions free the torso and lift onto gravity comp, and `whole_body` lets the base *admit* the net external wrench and glide — through `safe_base_step`, like every other twist). The bias comes from the region dynamics oracles in `dynamic_models/` (with the momentum observer estimating external torques). Entering a torque mode neutralises the position servos on the managed joints; exit restores them; every switch passes through `sync_to_current()` so nothing snaps. The Day-1 position PD (`position_controller/`, servo-inherited gains) remains the fallback law.

What must *not* change with the mode. The two paths above are different reference *generators*, and switching between them is the operator choosing an input device — not a different robot. Concretely: gravity compensation is one shared layer (`dynamic_models/gravity.py`) held by both, covering the same 22 actuated upper-body DoFs in every space, with the torque engine feeding exactly the complement of its managed set. When that was two implementations instead of one, leaving Drive dropped the feedforward on the ORCA wrists, neck, torso and lift, and repeated space changes walked the operator's palm 26.5 mm down; see `whole_body_controller.tex`, §"Mode-invariant gravity compensation". The general rule for this stack: if a quantity describes the *robot*

rather than the *operator's intent*, exactly one component should own it, and every mode should read it from there.

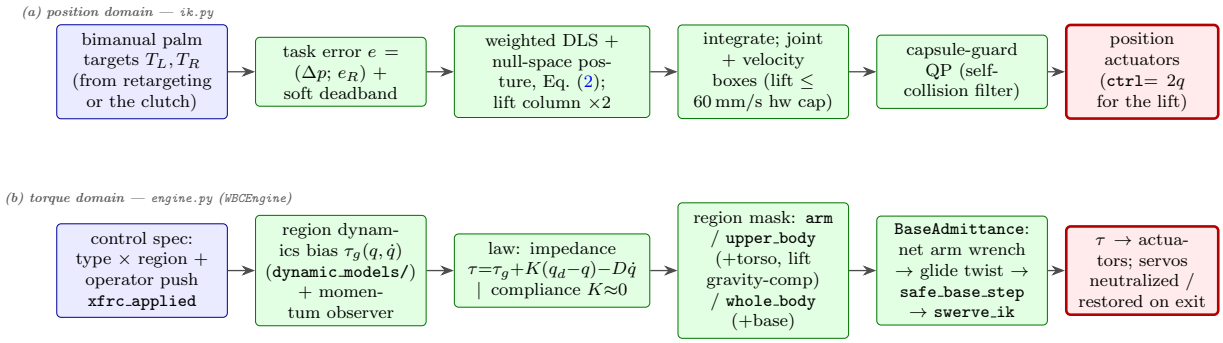


Figure 4: The two differential controllers. (a) Bimanual SE(3) differential IK: weighted damped least squares with a null-space posture task, deadbanded error, lift-unit doubling, and the capsule-guard QP as the final filter. (b) The torque runtime: type × region select the law; in `whole_body` the base admits the operator's push and glides — through the same tip-over gate as every twist.

6 Retargeting: operator anatomy to robot joints

Fig. 5 shows the two decoupled retargeting chains (papers: `hand_retargeting.tex`, `whole_body_retargeting.tex`, `body_frame.tex`, `calibration.tex`).

Whole body. The head and wrist poses are first expressed in a yaw-free operator frame, scaled by the per-operator calibration (a 3-pose anthropometry capture sets `REACH_SCALE`, the shoulder anchors and finger sizing). The arm map is *extension-fraction* based: the operator's fractional reach (wrist distance from shoulder over calibrated arm length) is reproduced on the robot's reach along the body-frame direction, which makes the map invariant to base yaw — you can walk and turn while teleoperating. The elbow swivel angle is taken from palm roll, and the resulting palm targets are solved by an analytic Gauss–Newton IK (33 → 5 ms after the 2026-07 rewrite) with upper-arm capsules and a separation floor keeping the arms apart. The neck is a decoupled look-at; the base, lift, and torso are *lazy followers* chasing head displacement and height through `safe_base_step`. The analytic body-frame estimator (`body_frame_estimator.py`) splits head+wrist motion into a *global* component (intended base drive/turn) and *local* manipulation — validated 7/7 canonical motions $\leq 2.1°$, 0% vs 98% false-drive against the twist-fit baseline — and is a drop-in at the engine's base-yaw hook, pending the physical-VP sign-off of the robot-frame sign.

Hands. Each ARKit 27-joint skeleton is converted to the 21-landmark layout, and the default *keypoint IK* retargeter solves the 17-DOF ORCA hand so its fingertip keypoints match the scaled operator keypoints; `vector.py` (fingertip direction matching, cf. DexPilot-style vector costs) and `anatomical.py` (per-joint angle transfer) are selectable alternatives, and `grasp.py` adds a grasp-closure assist. Hands are position-controlled and fully decoupled from the body's torque modes — a compliance push on the arm never opens the fingers.

7 Implementation and usage

- **Code.** The controller is the installable package `pip install -e controller/` (import `mabel`); the gateway is `server/`; the HAL is `hardware_bridge/` (`mabel_hw`); the sim twin is

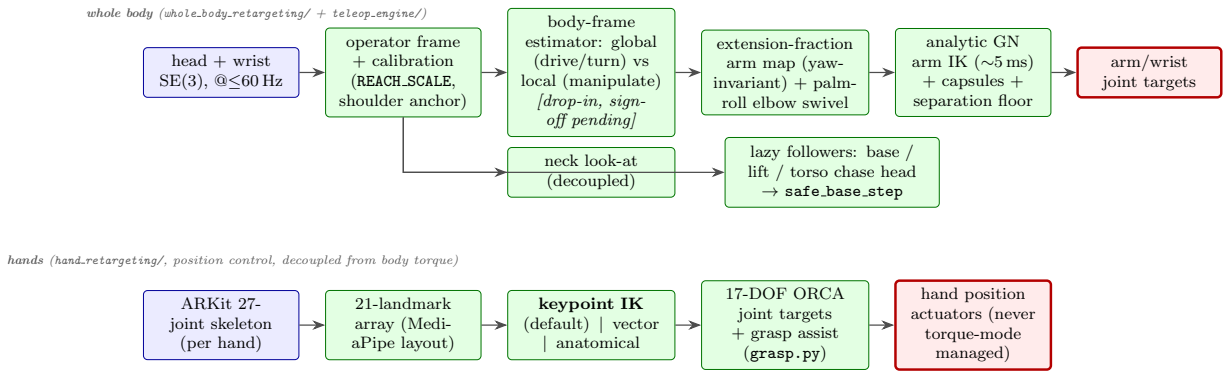


Figure 5: Retargeting. Top: the whole-body chain — calibrated operator frame, global/local split, yaw-invariant extension-fraction arm map, analytic GN IK, with the neck decoupled and the base/lift/torso as lazy followers behind the tip-over gate. Bottom: the hand chain — keypoint IK onto the 17-DOF ORCA hand, position controlled and independent of the body's torque modes.

`simulation_bridge/`.

- **Run.** Sim: `mjpython simulation/mabel_mujoco/scripts/teleop.py` (desktop viewer) or the studio `run.sh`; real: `server/real_run.sh` on thor (execs the gateway with `MABEL_DISPATCH_TARGET=real:ros`).
- **This document.** `make -C controller/docs` rebuilds `control_stack.pdf`; the diagrams are TikZ, generated from no data — when wiring changes, update Fig. 1 and the HAL contract tests.
- **Re-sync rule.** The motion-model constants in Fig. 3 are measured from the MJCF; after any robot-geometry change run `python3 controller/experiments/wbc_stability.py` (E4) and paste the printed values into `motion_model/tip_over.py`.

8 Limitations

This atlas is a wiring diagram, not a proof: rate numbers (50/60 Hz, 30 Hz gravity comp, 5–16 substeps) are the configured values, and the measured behaviour (loop pacing on thor, memoization hit rates, relay latency) lives in the experiment harnesses cited by the sibling papers. The body-frame estimator is drawn in the whole-body chain but is not yet live-wired (base-yaw sign needs the physical Vision Pro verification). The older research stack under `simulation/mabel_mujoco/scripts/teleop/` still coexists with the shipped package for the cosine-SLSQP arm variant; `mabel_controller.tex` flags which implementation each figure describes.

References

[1] B. Siciliano and O. Khatib (eds.), *Springer Handbook of Robotics*, 2nd ed., Springer, 2016. (Differential IK, task-priority and null-space control.)

[2] S. Chiaverini, B. Siciliano, and O. Egeland, "Review of the damped least-squares inverse kinematics with experiments on an industrial robot manipulator," *IEEE Trans. Control Syst. Technol.*, 2(2), 1994.

[3] Y. Nakamura, H. Hanafusa, and T. Yoshikawa, "Task-priority based redundancy control of robot manipulators," *Int. J. Robotics Research*, 6(2), 1987.

[4] M. Vukobratović and B. Borovac, "Zero-moment point — thirty five years of its life," *Int. J. Humanoid Robotics*, 1(1), 2004.

[5] A. Handa *et al.*, "DexPilot: Vision-based teleoperation of dexterous robotic hand-arm system," *IEEE ICRA*, 2020. (Fingertip/vector-cost hand retargeting.)